PhishGuard Complete Documentation

(Group 11A) .md

PhishGuard - 50% Implementation Report Content

1.1 Background

Phishing attacks represent one of the most prevalent cybersecurity threats in the modern digital landscape, targeting millions of users worldwide through deceptive websites designed to steal sensitive information such as login credentials, financial data, and personal details. These attacks have evolved significantly over the years, employing sophisticated techniques that make them increasingly difficult to detect through traditional means. Attackers often create fraudulent websites that closely mimic legitimate platforms, using similar domain names, visual designs, and URL structures to deceive unsuspecting users.

The sophistication of phishing attacks has necessitated the development of advanced detection mechanisms that can identify malicious websites in real-time, before users interact with them. Traditional blacklist-based approaches, while effective against known threats, fail to protect users from newly created phishing sites that have not yet been catalogued. This limitation has led to the exploration of heuristic-based detection systems that analyze URL characteristics and webpage content patterns to identify suspicious websites proactively.

Browser extensions have emerged as an ideal platform for implementing such protection mechanisms, as they can integrate seamlessly with users' browsing experience, providing real-time warnings and visual indicators without disrupting normal web navigation. Modern browsers, particularly Chrome with its Manifest V3 architecture, provide powerful APIs that enable extensions to monitor web navigation, analyze page content, and implement security checks at the network layer.

1.2 Motivation

We aim to tackle the rise of sophisticated phishing attacks by developing an AI system that detects subtle visual and QR-based threats, alerting users and protecting their privacy and online security.

1.3 Objective

The project aims to develop an AI system that detects malicious websites, phishing images, QR codes, and fake pop ups, alerting users in real time to protect their data and enhance cybersecurity with a user-friendly interface.

1.4 Report Outline

This report is structured to provide a comprehensive overview of the PhishGuard project's current implementation status, covering both the theoretical foundations and practical implementation details of the system developed to date. The document begins with an introduction that establishes the background, motivation, and objectives of the project, setting the context for the technical discussions that follow.

Section 2, "Study of the System," examines the core techniques employed in the implementation, exploring the hybrid detection methodology that combines URL heuristic analysis with machine learning approaches. This section discusses the various available techniques for phishing detection, compares their strengths and limitations, and reviews related work in the field to situate this project within the broader context of cybersecurity research and development.

Subsequent sections will detail the system architecture, implementation specifics of the Chrome extension and machine learning backend, evaluation methodologies, and findings from testing and validation efforts. The report concludes with a discussion of current limitations, future work planned for the remaining project phases, and conclusions drawn from the implementation completed thus far.

2.1 About the Technique

The PhishGuard system employs a hybrid detection technique that synergistically combines rule-based URL heuristic analysis with machine learning-powered webpage content examination. This two-tiered approach addresses the complementary strengths and weaknesses of each method, resulting in a more robust and accurate detection system than either approach could achieve independently.

The first tier utilizes URL heuristic analysis, which examines various structural and lexical characteristics of website URLs to identify potentially suspicious patterns. This includes detection of IP addresses used directly as domains, excessive subdomain nesting, hyphens within domain names, punycode encoding that might hide malicious characters, and the presence of suspicious keywords such as "login," "verify," "update," and "secure" within URL paths. These heuristics are based on well-documented patterns observed in phishing attacks, where attackers frequently employ domain manipulation techniques to create URLs that appear legitimate while redirecting to malicious servers.

The second tier incorporates machine learning analysis, specifically employing a Random Forest Classifier trained on a diverse dataset of over 11,000 legitimate and suspicious websites. The machine learning component analyzes 18 distinct features extracted from both URL structures and webpage HTML content, including URL length, domain characteristics, presence of login forms, password input fields, external link counts, image counts, and frequency of suspicious keywords within webpage content. This multi-dimensional feature analysis allows the model to identify subtle patterns and correlations that may not be immediately apparent through rule-based methods alone.

The integration of these techniques creates a complementary detection system where URL heuristics provide immediate, deterministic classification for obvious threats, while machine learning offers probabilistic analysis for edge cases and sophisticated attacks that may evade simple pattern matching. The system's decision-making logic allows either detection method to independently flag a website as suspicious, ensuring comprehensive coverage while maintaining high recall rates for potential threats.

2.2 Various Available Techniques

The field of phishing detection encompasses several distinct methodologies, each with unique advantages and limitations that influence their effectiveness in different scenarios. Understanding these various techniques is crucial for designing a comprehensive detection system and appreciating the rationale behind the hybrid approach adopted in this project.

Blacklist-Based Detection is among the oldest and most widely implemented methods, relying on curated databases of known malicious URLs maintained by security organizations such as Google Safe Browsing, Phishtank, and OpenPhish. This approach offers high precision for identified threats and minimal false positives, but suffers from the fundamental limitation of being reactive rather than proactive—it can only detect threats that have already been discovered and catalogued. The time delay between a phishing site's creation and its addition to blacklists leaves users vulnerable to zero-day attacks, and maintaining comprehensive, up-to-date lists requires significant resources and constant monitoring.

Heuristic-Based Detection employs rule-based algorithms that analyze URL structure, domain characteristics, and lexical patterns to identify suspicious indicators without requiring prior knowledge of specific malicious sites. This approach can detect previously unknown threats by recognizing common attack patterns, such as typo-squatting (using domains like "paypa1.com" instead of "paypal.com"), subdomain manipulation, and keyword stuffing.

However, heuristic systems may generate false positives when legitimate websites employ similar patterns, and sophisticated attackers can potentially craft URLs that evade heuristic rules while still serving malicious content.

Machine Learning-Based Detection leverages artificial intelligence algorithms to learn patterns from large datasets of labeled examples, enabling the system to generalize and identify phishing characteristics beyond explicitly programmed rules. Techniques such as Random Forest, Support Vector Machines, Neural Networks, and Deep Learning have been applied to phishing detection, with features extracted from URLs, webpage content, and even behavioral patterns. Machine learning approaches can adapt to evolving attack techniques and identify subtle patterns, but they require substantial training data, may struggle with interpretability, and can be susceptible to adversarial examples designed to fool the model.

Content-Based Analysis examines the actual HTML, JavaScript, and visual elements of webpages to identify suspicious characteristics such as form structures, embedded scripts, redirect mechanisms, and visual similarity to legitimate sites. This approach can catch sophisticated attacks that use legitimate-looking domains but serve malicious content, but it requires fetching and parsing webpages, introducing latency and computational overhead.

Hybrid Approaches, such as the one implemented in PhishGuard, combine multiple techniques to leverage their complementary strengths. By integrating URL heuristics with machine learning content analysis, hybrid systems can achieve both the speed of rule-based detection and the adaptability of learned models, providing comprehensive protection while mitigating individual method limitations.

2.3 Related Works

The problem of phishing detection has been extensively researched in both academic and industry contexts, with numerous approaches and systems developed to address various aspects of the threat landscape. Understanding related work helps contextualize the PhishGuard implementation and identify both the contributions of this project and areas for future enhancement.

Academic Research in phishing detection has explored various machine learning algorithms and feature sets. Studies have investigated the effectiveness of different classifiers, with Random Forest, Support Vector Machines, and Neural Networks showing promise for binary classification tasks. Research has also examined optimal feature selection, identifying that combinations of URL-based features (length, subdomain count, special characters) and content-based features (form presence, external links, keyword frequency) provide superior accuracy compared to either category alone. However, many academic approaches focus on offline analysis rather than real-time browser integration, limiting their practical applicability.

Industry Solutions include browser-integrated systems such as Google Safe Browsing, which uses a combination of blacklists and heuristics to warn users about potentially dangerous sites. Microsoft's SmartScreen and Firefox's built-in protection systems employ similar approaches, primarily relying on reputation-based blacklists with supplementary heuristic checks. While effective, these systems are proprietary, limiting transparency and customization, and may not provide the granular control desired for specialized use cases or research applications.

Browser Extension Solutions such as Netcraft's Anti-Phishing Toolbar, Web of Trust (WOT), and various academic research prototypes have demonstrated the feasibility of client-side phishing detection. These solutions vary widely in their approaches—some rely primarily on community reporting and reputation systems, while others implement technical analysis similar to PhishGuard's methodology. However, many existing extensions face challenges with accuracy, user experience, or scalability, and few successfully integrate both heuristic and machine learning components in a seamless, real-time detection framework.

Recent Developments in the field have increasingly focused on deep learning techniques, using Convolutional Neural Networks for visual similarity detection and Natural Language Processing for analyzing webpage text content. While promising, these approaches require significant computational resources and may not be suitable for real-time browser extension deployment without cloud backend support.

The PhishGuard project contributes to this landscape by demonstrating a practical, integrated approach that combines URL heuristics with machine learning content analysis within a lightweight browser extension framework, providing real-time protection without requiring cloud dependencies for basic functionality. The system's four-label classification scheme (safe, unknown, suspicious, malicious) offers more granular threat assessment than binary approaches, enabling users to make informed decisions about website trustworthiness.

3.1 Problem Statement

The proliferation of phishing attacks presents a significant and growing cybersecurity challenge, with attackers continuously evolving their techniques to bypass traditional detection mechanisms. The core problem addressed by this project is the inadequacy of existing protection systems that rely on single-method approaches, which are either too reactive (blacklists) or too prone to false positives (pure heuristics), or too resource-intensive (standalone machine learning models) for real-time browser-based deployment.

Current solutions suffer from several critical limitations: blacklist-based systems cannot protect against zero-day phishing attacks until threats are identified and catalogued, creating a window of vulnerability that attackers exploit. Heuristic-based systems, while faster, often generate false alarms when legitimate websites employ similar structural patterns to phishing sites, leading to user frustration and potential bypassing of security warnings. Standalone machine learning implementations typically require cloud connectivity, raising privacy concerns and introducing latency that degrades user experience.

Additionally, the existing landscape lacks a unified system that provides granular threat assessment—most solutions offer binary safe/unsafe classifications that do not account for the spectrum of risk levels that websites may exhibit. Users require actionable information that distinguishes between confirmed malicious sites, suspicious sites requiring caution, and unknown sites that have not yet been evaluated, enabling informed decision-making rather than blanket blocking or ignoring.

The PhishGuard project addresses these problems by proposing an integrated, hybrid detection system that combines the speed of heuristic analysis with the intelligence of machine learning, operating locally within a browser extension to provide real-time, privacy-preserving protection that adapts to evolving attack patterns without requiring constant cloud connectivity for basic functionality.

3.2 Scope

The scope of the PhishGuard project encompasses the development of a comprehensive phishing detection system implemented as a Chrome browser extension, integrated with a machine learning backend for enhanced threat identification. The current implementation phase focuses on establishing core functionality through hybrid detection methodologies, setting the foundation for future enhancements that will expand system capabilities and effectiveness.

In-Scope Components include: (1) development of a Chrome extension using Manifest V3 architecture that monitors web navigation and classifies websites into four distinct categories—safe, unknown, suspicious, and malicious—based on URL characteristics and webpage content analysis; (2) implementation of a rule-based URL heuristic engine that detects suspicious patterns including IP addresses, excessive subdomains, hyphens, punycode encoding, and keyword manipulation; (3) development of a machine learning model using Random Forest classification, trained on datasets of legitimate and phishing websites, capable of analyzing 18 distinct features extracted from both URLs and webpage HTML content; (4) creation of a Python Flask API backend that serves machine learning predictions to the extension in real-time; (5) design and implementation of user interface components including dynamic toolbar icons, popup warnings, full-screen interstitials for malicious sites, and configuration options for whitelist/blacklist management; and (6) integration of dark/light theme support and user preference management using browser storage APIs.

Out-of-Scope Elements for the current phase include: (1) cloud-based backend infrastructure deployment, which will be addressed in future phases; (2) community moderation and user reporting systems, planned for implementation in subsequent development cycles; (3) integration with external threat intelligence feeds such as Phishtank or Google Safe Browsing APIs, scheduled for Phase 4; (4) advanced machine learning techniques such as deep learning or neural networks, which may be explored in future optimization phases; and (5) multi-browser support beyond Chrome, though the architecture is designed to be extensible to other browsers.

The project scope is intentionally focused on establishing a functional, accurate, and user-friendly core system that demonstrates the viability of hybrid detection approaches, with clear pathways for expansion and enhancement in subsequent development phases.

3.3 Proposed System

The PhishGuard system is proposed as an intelligent, multi-layered phishing detection solution that operates seamlessly within the Chrome browser environment, providing real-time protection through a hybrid methodology that integrates rule-based URL heuristic analysis with machine learning-powered webpage content examination. The system architecture is designed to be lightweight, privacy-preserving, and extensible, enabling continuous improvement through model retraining and feature enhancement without disrupting user experience.

The proposed system consists of three primary components: (1) a Chrome extension frontend that handles user interface presentation, browser event monitoring, and local classification using URL heuristics; (2) a machine learning backend implemented as a Python Flask API server that provides probabilistic threat assessment based on comprehensive feature analysis of webpage content; and (3) a classification engine that synthesizes inputs from both heuristic rules and machine learning predictions to assign websites to one of four risk categories, with corresponding visual indicators and user warnings.

The detection workflow begins when a user navigates to a website, triggering the extension's background service worker to extract the URL and perform initial heuristic analysis. This rapid, deterministic evaluation can immediately classify websites as safe (if present in a curated whitelist of known legitimate domains) or identify obvious suspicious patterns. For websites classified as unknown or suspicious by heuristic rules, the system queries the machine learning API, which fetches the webpage content, extracts 18 features encompassing URL structure and HTML characteristics, and generates a probabilistic prediction regarding the site's maliciousness.

The proposed system's decision-making logic employs a complementary approach where either detection method can independently flag a website as suspicious—URL heuristics provide immediate classification for obvious threats, while machine learning offers nuanced analysis for edge cases. This dual-layer protection ensures comprehensive coverage: websites with suspicious URL patterns are flagged even if webpage content appears benign, and sophisticated attacks using legitimate-looking URLs are caught through content analysis. The four-label classification scheme (safe, unknown, suspicious, malicious) provides users with actionable information, distinguishing between confirmed threats requiring blocking, suspicious sites warranting caution, and unknown sites requiring further evaluation.

User interaction is facilitated through intuitive visual indicators: dynamic toolbar icons change color based on classification (green for safe, yellow for suspicious, red for malicious, gray for unknown), popup interfaces provide detailed threat information and action options, and full-screen interstitials block access to confirmed malicious sites with options to proceed, go back, or close the tab. The system maintains user preferences including whitelist/blacklist configurations and theme settings, ensuring personalized protection that respects user autonomy while providing robust security guidance.

4.1 Requirement Engineering

Requirement engineering for the PhishGuard project involved systematic identification, analysis, and documentation of functional and non-functional requirements essential for delivering an effective phishing detection system. This process ensured that the system addresses real-world security needs while maintaining usability, performance, and extensibility required for practical deployment and future enhancement.

4.1.1 Requirement Elicitation

Requirement elicitation was conducted through analysis of existing phishing detection solutions, review of cybersecurity research literature, and identification of gaps in current protection mechanisms. The process identified key stakeholder needs including: end-users requiring real-time, non-intrusive protection that does not disrupt normal browsing activities; security researchers needing transparent, extensible systems for threat analysis; and system administrators requiring lightweight, privacy-preserving solutions that do not introduce significant performance overhead.

Functional requirements derived from this analysis include: the system must classify websites into four categories (safe, unknown, suspicious, malicious) based on URL and webpage characteristics; it must provide visual indicators (toolbar icons) that update dynamically based on classification results; it must display warning interfaces for suspicious and malicious sites with actionable options; it must support user-configurable whitelists and blacklists; it must integrate machine learning predictions for enhanced accuracy; and it must operate in real-time without noticeable latency affecting page load times.

Non-functional requirements identified encompass: performance requirements specifying that URL heuristic classification must complete within milliseconds, and machine learning API calls should not exceed 2-3 seconds; usability requirements mandating intuitive interfaces that do not require technical expertise; privacy requirements ensuring that webpage content analysis occurs locally or through user-controlled endpoints; reliability requirements specifying graceful degradation when ML backend is unavailable; and extensibility requirements enabling future integration with threat intelligence feeds and community moderation systems.

4.1.2 Software Lifecycle Model

The PhishGuard project adopts an iterative, incremental software development lifecycle model, specifically utilizing aspects of the Agile methodology combined with elements of the Spiral model to accommodate both rapid prototyping of core features and systematic risk management for security-critical components. This approach allows for continuous refinement based on testing feedback while maintaining structured phases for requirement validation, architecture design, implementation, and evaluation.

The current implementation represents the first major increment, focusing on establishing core functionality including URL heuristic classification, basic machine learning integration, and user interface components. Subsequent increments will introduce backend server infrastructure, community moderation capabilities, and advanced model optimization. Each increment follows a cycle of planning, design, implementation, testing, and evaluation, with user feedback informing priorities for the next iteration.

Risk-driven elements from the Spiral model are incorporated through systematic evaluation of security implications at each phase—ensuring that machine learning model accuracy is validated before deployment, that API security measures are implemented to prevent abuse, and that user privacy is protected throughout the system architecture. This hybrid lifecycle approach balances the flexibility needed for rapid development with the rigor required for security-critical applications.

4.1.3 Requirement Analysis

Requirement analysis involved detailed examination of elicited requirements to ensure feasibility, consistency, and traceability, identifying dependencies between functional components and establishing verification criteria for system validation.

4.1.3.1 UML Diagrams/DFDs Based on the Project

The system architecture can be represented through multiple diagrammatic models: **Use Case Diagrams** illustrate interactions between actors (users, browsers, ML backend) and system components (classification engine, UI components, API server), showing use cases such as "Classify Website," "Display Warning," "Manage Whitelist," and "Query ML API." **Sequence Diagrams** depict the flow of classification requests from browser navigation events through heuristic analysis, ML API calls, and UI updates, demonstrating temporal interactions between background service worker, content scripts, popup interface, and Flask backend.

Data Flow Diagrams (DFDs) model information flow through the system: Level 0 DFD shows external entities (User, Web Browser, ML Backend) exchanging data with the PhishGuard system; Level 1 DFD decomposes the system into processes including "Monitor Navigation," "Extract URL Features," "Perform Heuristic Analysis," "Query ML Backend," "Synthesize Classification," "Update UI," and "Manage User Preferences," with data stores for whitelists, blacklists, user settings, and training data. **Component Diagrams** represent the structural organization showing relationships between Chrome extension modules (background.js, popup.js, content scripts), classifier components, ML backend modules (Flask app, feature extraction, model inference), and external dependencies.

4.1.3.2 Cost Analysis

Cost analysis for the PhishGuard project encompasses development costs, operational expenses, and infrastructure requirements. **Development Costs** include time investment for implementation (estimated at 200-300 hours for current phase), tooling and software licenses (Chrome Web Store developer account: $5 one-time fee; development tools: free/open-source), and training data acquisition (public datasets: free; potential commercial datasets: variable). **Operational Costs** for the current local deployment model are minimal—users run the ML backend on their own machines, requiring only standard hardware resources.

Future Infrastructure Costs (for planned cloud deployment) include cloud hosting services (estimated $20-50/month for small-scale deployment on platforms like Heroku, AWS, or Google Cloud), database storage for threat intelligence feeds (estimated $10-30/month), and API usage fees if integrating with commercial threat intelligence services (varies by provider and usage volume). **Maintenance Costs** involve ongoing model retraining with updated datasets (estimated 10-20 hours monthly), bug fixes and feature enhancements (estimated 5-10 hours monthly), and monitoring and optimization of ML model performance.

The total cost for the current 50% implementation phase is primarily development time, with minimal monetary expenses. Future phases will require cloud infrastructure investment, but the modular architecture allows for gradual scaling based on user adoption and resource availability.

4.1.3.3 Hardware and Software Requirements

Hardware Requirements for running PhishGuard are minimal, designed to function on standard consumer hardware: users require a computer with at least 2GB RAM (4GB recommended for optimal ML backend performance), a modern multi-core processor (2+ cores recommended for parallel data processing during model training), and standard network connectivity for fetching webpage content and, optionally, communicating with cloud-based threat intelligence feeds. Storage requirements are modest: approximately 50-100MB for the Chrome extension files, 200-500MB for Python ML backend dependencies, and 100-500MB for training datasets and model files.

Software Requirements include: Chrome browser version 88 or higher (supporting Manifest V3), Python 3.8 or higher for ML backend execution, and standard Python packages including Flask, scikit-learn, pandas, numpy, BeautifulSoup4, and requests. Development requirements include a code editor (VS Code, Sublime Text, or similar), Git for version control, and Chrome Developer Tools for extension debugging. The system is designed to be cross-platform, compatible with Windows, macOS, and Linux operating systems.

Browser API Requirements encompass Chrome extension APIs including: `chrome.tabs` for tab monitoring and URL extraction, `chrome.storage` for local data persistence, `chrome.scripting` for content script injection, and

`chrome.webNavigation` for navigation event tracking. The extension requires host permissions for `<all_urls>` to enable comprehensive website analysis, and storage permissions for maintaining user preferences and classification history.

4.2 System Architecture

The PhishGuard system architecture follows a modular, distributed design that separates concerns between client-side browser extension components and server-side machine learning processing, enabling efficient real-time threat detection while maintaining privacy and performance objectives. The architecture is structured to support both local ML backend operation (current implementation) and future cloud-based deployment, ensuring flexibility and scalability.

4.2.1 UI/UX Diagram

The user interface architecture is organized into distinct components that provide seamless interaction across different contexts: **Toolbar Icon** serves as the primary visual indicator, dynamically updating color (green/yellow/red/gray) based on website classification, and providing access to detailed information through click interaction. **Popup Interface** displays comprehensive site classification details when users click the toolbar icon, showing current website status, classification reasons, threat level indicators, and actionable buttons including "Proceed Anyway," "Go Back," "Report Site," and "Close Tab" (context-dependent based on threat level).

Full-Screen Interstitial appears for confirmed malicious sites, blocking page access entirely and presenting a prominent warning message with options to navigate away, proceed at user's own risk, or close the tab. **Suspicious Site Banner** provides a non-intrusive, dismissible notification for suspicious sites, allowing users to acknowledge the warning while maintaining page accessibility. **Options Page** enables users to manage whitelisted and blacklisted domains, configure theme preferences (dark/light mode), and view extension settings.

The UI/UX design prioritizes information hierarchy: critical threats (malicious) receive full-screen blocking with prominent warnings, moderate threats (suspicious) receive visible but non-blocking notifications, and safe/unknown classifications provide subtle visual feedback without interrupting browsing. Color coding follows intuitive conventions: green for safe, yellow for caution (suspicious), red for danger (malicious), and gray for unclassified (unknown). The interface maintains consistency across light and dark themes, with CSS variables enabling seamless theme switching that syncs across all UI components including content script overlays and popup interfaces.

User experience flow follows a progressive disclosure model: toolbar icons provide immediate visual feedback, popup interfaces offer detailed information on demand, and interstitials provide comprehensive warnings only when necessary. This design minimizes cognitive load while ensuring users receive appropriate security guidance based on threat severity, balancing protection effectiveness with browsing convenience.

(Group 11A) 75%_PROGRESS.md

PhishGuard - 75% Implementation Progress Report

1. Executive Summary

PhishGuard is an advanced browser extension designed to protect users from phishing attacks in real-time. Unlike traditional solutions that rely solely on blacklists or simple heuristics, PhishGuard employs a **hybrid detection mechanism** combining local rule-based analysis with a cloud-hosted Machine Learning (ML) backend.

This report documents the status of the project at the **75% completion milestone**. All core components—including the Chrome Extension frontend, the Flask/ML backend, and the real-time integration logic—are fully implemented,

trained, and deployed. The remaining work focuses on rigorous testing, validation, and community-driven features.

---

2. System Architecture

The system follows a distributed client-server architecture:

2.1 Frontend: Chrome Extension (Manifest V3)

Running locally in the user's browser, the extension serves as the first line of defense.

- **Service Worker (`background.js`)**: Orchestrates detection logic. It first runs a quick heuristic check; if the site's status is inconclusive ("unknown" or "suspicious"), it asynchronously queries the ML API.
- **Content Scripts**:
  - `content.js` : Injects a full-screen blocking overlay for confirmed malicious sites.
  - `content_suspicious.js` : Injects a non-intrusive warning badge for suspicious sites.
- **UI Components**:
  - **Popup**: Provides detailed site status, confidence scores, and user actions ("Trust Site", "Report", "Proceed").
  - **Dynamic Icons**: Visual indicators (Green/Yellow/Red/Gray) in the browser toolbar.

2.2 Backend: Machine Learning API

Hosted on **Railway**, providing intelligence for complex threats.

- **API Server (`app.py`)**: A Python Flask application exposing `/predict` and `/report` endpoints.
- **ML Model**: A Random Forest Classifier trained on ~11,500 URLs (legitimate vs. phishing) with ~84% initial accuracy.
- **Feature Extraction**: Analyzes 18 distinct features per URL, including:
  - URL structure (length, subdomains, special characters, IP addresses).
  - HTML content (presence of login forms, password fields, suspicious keywords).

---

3. Implementation Status (75% Complete)

✅ Completed Modules

1. Core Detection Engine

- **URL Heuristics**: Algorithms to detect common phishing tricks (e.g., IP usage, punycode, excessive subdomains, keyword stuffing).
- **Machine Learning Integration**: Full automated pipeline to unseen URLs to the backend for content analysis.
- **Hybrid Logic**: Smart decision-making that prioritizes known-safe lists (whitelists) while using ML to catch novel zero-day threats.

2. Machine Learning Pipeline

- **Data Collection (`collect_data.py`)**: A multi-threaded scraper that builds datasets from real-world URLs.
- **Training (`train_model.py`)**: Automated training script that generates the `phishguard_model.pkl` file.
- **Deployment**: The backend is containerized and live on Railway, accessible via HTTPS.

3. Deployment Workflow (Railway & GitHub)

The system effectively uses a Continuous Deployment (CD) pipeline:

1. **Version Control**: All code (frontend + backend) is hosted on GitHub (`G-man312/Phishguard-`).
2. **Railway Integration**:
   - We connected the Railway project directly to the GitHub repository.
   - Configured the **Root Directory** to `ml-backend` so Railway detects the `Dockerfile` and Python requirements automatically.
3. **Build Process**:
   - On every `git push` to `main`, Railway triggers a new build.
   - It installs dependencies from `requirements.txt` and starts the Flask server using `gunicorn`.
4. **Live Endpoint**: Railway assigns a permanent SSL-enabled domain (e.g., `https://phishguard-production-f4cc.up.railway.app`) which is hardcoded into the Chrome Extension's `background.js`.

4. User Interface & Experience

- **Real-time Feedback**: Status badges update instantly on tab navigation.
- **Safety Forcefields**: Interstitials block user access to dangerous sites until they explicitly choose to proceed.
- **Customization**: Users can toggle Dark/Light modes and manage their own local whitelist.

5. Reporting System

- **User Reporting**: Users can flag false positives or new phishing sites directly from the extension.
- **Data Persistence**: Reports are logged to the backend (`reports.md`) for future model retraining.

---

4. Technical Highlights

- **Performance**: The "Quick Check" (heuristics) runs in milliseconds locally. ML queries are only made when necessary to preserve privacy and bandwidth.
- **Resilience**: If the ML backend is offline, the extension falls back to heuristic-only mode gracefully.
- **Accuracy**: The Random Forest model leverages both "lexical" (URL text) and "content" (HTML body) features, significantly reducing false positives compared to URL-only models.

---

5. Remaining Work (Final 25%)

The final phase of the project is dedicated to **Testing, Validation, and Refinement**:

1. **Comprehensive Testing**:
   - Unit tests for individual heuristic functions.
   - End-to-end integration tests ensuring frontend-backend sync.
   - False positive reduction analysis.
2. **Community Features**:
   - Enhancing the reporting system to feed directly into a retraining pipeline.
   - Verified "Community Trust" lists.
3. **Documentation & Polish**:
   - Finalizing the user manual and developer guide.
   - Preparing for Chrome Web Store submission (privacy policy, assets).

---

6. Conclusion

At 75% progress, PhishGuard is a fully functional security tool. It successfully demonstrates the power of combining traditional rule-based security with modern Machine Learning techniques to protect users from the evolving landscape of web threats.

CONCLUSION_FUTURE_SCOPE.md

Conclusion and Future Scope

6. Conclusion

This report presents a study of different phishing detection techniques, including URL heuristic analysis, machine learning-based classification, and hybrid approaches. The study explores various domain techniques including URL pattern analysis (IP address detection, subdomain analysis, punycode identification), content-based filtering (HTML content analysis, form detection, keyword extraction), and machine learning classification using Random Forest algorithm. A hybrid approach is proposed that combines URL heuristic-based detection with machine learning predictions, where the system first performs quick URL characteristic checks and then augments the classification with ML model confidence scores. A comparative study of various techniques mentioned above is presented, demonstrating that the hybrid approach achieves superior performance compared to standalone heuristic or ML-based methods. The performance measures used to evaluate the system include Accuracy, Precision, Recall, and F1-Score, which are described and analyzed throughout the report. Different standard datasets or variable inputs are defined for experiments, with the primary dataset being the URL dataset containing 450,000+ URLs (104,438 phishing and 345,738 legitimate URLs) sourced from PhisTank, Majestic Million, and other reputable sources. Additionally, custom safe and malicious domain datasets are maintained for real-time classification. The applications of this domain are identified and presented, including real-time browser protection through a Chrome extension, user reporting mechanisms for suspicious sites, Flask API backend for ML predictions, and seamless integration with web browsing workflows to provide immediate visual feedback and security warnings.

7. Future Scope

The future scope of this phishing detection system encompasses several promising directions for enhancement and expansion. Community-based moderation can be implemented where users can report suspicious sites, and verified community moderators can review and validate reports, creating a crowdsourced threat intelligence database that continuously improves the system's detection capabilities. Advanced machine learning techniques can be integrated, including deep learning models such as CNNs for URL pattern recognition and LSTM networks for sequential URL analysis. The system can be extended to support real-time dataset updates through integration with threat intelligence feeds such as PhisTank API and OpenPhish. Cross-browser compatibility can be implemented by developing extensions for Firefox, Microsoft Edge, Safari, and Opera. Cloud-based detection services can be deployed to provide centralized threat analysis, allowing multiple users to benefit from collective threat intelligence. Mobile application development for Android and iOS platforms would extend the protection to smartphone users. Enhanced feature extraction can incorporate SSL certificate analysis, DNS-based reputation checking, WHOIS data analysis, and website screenshot analysis using computer vision techniques. Real-time collaborative learning can be implemented where user reports contribute to model retraining, creating a continuously improving detection system that adapts to emerging phishing trends.

DEPLOY_GUIDE.md

PhishGuard Deployment Guide for Fly.io (Complete Walkthrough)

This guide walks you through deploying your PhishGuard ML backend to Fly.io **step by step**. Follow each section carefully.

Prerequisites

- You have a Fly.io account and logged into the web UI
- You created an empty GitHub repo and connected it to Fly
- You have Git installed on your computer
- You're in the project folder (`E:\(Group 11A) phish-guard\phish-guard`)

Step 1: Push Your Code to GitHub

1.1 Open PowerShell

Press `Win + X` and select **Windows PowerShell** (or open Terminal in VS Code).

1.2 Navigate to your project folder

```
cd "E:\(Group 11A) phish-guard\phish-guard"
```

Verify you're in the right place by checking if you see the `ml-backend` folder:

```
ls ml-backend
```

You should see: `app.py` , `requirements.txt` , `Dockerfile` , etc.

1.3 Configure Git (first time only)

If you haven't used Git before, run:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

1.4 Initialize Git repo (if not already done)

```
git init
```

1.5 Add all files to Git

```
git add .
```

1.6 Create initial commit

```
git commit -m "Initial commit: PhishGuard Chrome extension + ML backend"
```

Expected output: should show files being committed (something like "create mode 100644 manifest.json", etc.)

1.7 Add GitHub repo as remote

Replace `YOUR_USERNAME` and `YOUR_REPO` with your actual GitHub username and repo name:

```
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPO.git
```

Example:

```
git remote add origin https://github.com/G-man312/phishguard.git
```

1.8 Push to GitHub

```
git branch -M main
git push -u origin main
```

What to expect:

- PowerShell may prompt you to login to GitHub (a browser window will open)
- You'll see `Enumerating objects...`, `Compressing objects...`, then `done.`
- After ~30 seconds, you should see confirmation

Verify it worked:

- Go to your GitHub repo URL (e.g., `https://github.com/G-man312/phishguard`) in your browser
- You should see all your files (manifest.json, ml-backend folder, etc.)

Step 2: Configure Fly.io to Deploy from GitHub

2.1 Go to Fly.io Dashboard

1. Open https://fly.io in your browser
2. Log in if needed
3. Click the "+" icon or "Create App" button (top left area)

2.2 Connect GitHub Repo

1. A dialog appears: **"Choose a repository"**
2. On the **left side**, you'll see your GitHub account listed with your repos
3. **Find and click on your repo** (e.g., `G-man312/phishguard`)
    - You may see a "Refresh cache" button; click it if your repo doesn't appear

2.3 Configure Deployment Settings

After clicking your repo, a form appears with these fields:

Field	Value
App name	`phishguard` (or any unique name)
Organization	`Personal`
Branch to deploy	`main`
Current Working Directory	`ml-backend` (IMPORTANT!)

Config path	Leave empty

The most important setting: Set "Current Working Directory" to `ml-backend` because that's where your `Dockerfile` is.

2.4 Click "Deploy"

1. Fly will start building your Docker image (this takes 1-3 minutes)
2. You'll see a log output with lines like:

```
Building docker image...
Installing Python dependencies...
Successfully deployed
```

2.5 Wait for Deployment to Complete

- You'll see a green checkmark or "Success" message
- Fly will assign your app a URL like: `https://phishguard.fly.dev`
- **Copy and save this URL** — you'll need it later!

Step 3: Verify the Backend is Running

3.1 Test the `/health` endpoint

1. Open your browser
2. Go to: `https://<YOUR_FLY_APP>.fly.dev/health`
   - Replace `<YOUR_FLY_APP>` with your actual app name (e.g., `phishguard`)
   - Full example: `https://phishguard.fly.dev/health`
3. You should see JSON output like:

```
{
  "status": "healthy",
  "model_loaded": true
}
```

If you see this ✅ **Great! Your backend is deployed and working.**

If you see an error ❌ Go to **Troubleshooting** section at the end.

Step 4: Update the Extension to Use Your Deployed Backend

Now you need to tell the Chrome extension to use your new Fly.io URL instead of `localhost:5000`.

4.1 Open background.js in VS Code

1. In VS Code, open the file: `background.js`
2. Find the line that says:

```
const ML_API_URL = 'http://localhost:5000/predict';
```

(Should be around line 4)

3. Replace `http://localhost:5000` with your Fly.io URL:

```
const ML_API_URL = 'https://phishguard.fly.dev/predict';
```

(Replace `phishguard` with your actual Fly app name)

4. Save the file (Ctrl + S)

4.2 Open popup.js in VS Code

1. Open the file: `popup.js`
2. Find the lines that say:

```
const ML_API_URL = 'http://localhost:5000/predict';
const REPORT_API_URL = 'http://localhost:5000/report';
```

(Should be around lines 2-3)

3. Replace both lines:

```
const ML_API_URL = 'https://phishguard.fly.dev/predict';
const REPORT_API_URL = 'https://phishguard.fly.dev/report';
```

4. Save the file (Ctrl + S)

That's it! Your extension will now talk to your deployed backend on Fly.io.

Step 5: Test the Extension Locally (Before Publishing)

5.1 Load the Extension in Chrome

1. Open Chrome browser
2. Go to: `chrome://extensions/`
3. Toggle **"Developer mode"** (top right corner)
4. Click **"Load unpacked"** (top left)
5. Select your project folder: `E:\(Group 11A) phish-guard\phish-guard`
6. Click **"Select Folder"**

You should see the PhishGuard extension appear in the list with a colorful icon.

5.2 Test It

1. Click the PhishGuard icon in your Chrome toolbar (top right)
2. The popup should load and show "Checking..." briefly
3. Visit a website (e.g., `https://google.com`)
4. Click the PhishGuard icon again — you should see the classification (e.g., "Safe Site")

If this works ✅ Your extension is now using the Fly.io backend!

If it fails ❌ Check that your URL in `background.js` and `popup.js` matches your Fly.io app name exactly.

Step 6: Prepare for Chrome Web Store Publishing (Optional)

Once you're happy with testing, you can publish to the Chrome Web Store so anyone can install it with one click.

(I can provide detailed instructions for this if you need them.)

Troubleshooting

My `/health` endpoint returns an error

Cause: The Flask app is running, but the ML model file wasn't included in the Docker build.

Fix:

1. Make sure `phishguard_model.pkl` exists in `ml-backend` folder (check locally first)
2. Push it to GitHub:

   ```
   git add ml-backend/phishguard_model.pkl
   git commit -m "Add ML model file"
   git push origin main
   ```

3. Redeploy from Fly.io UI (find the "Redeploy" button)

The extension can't connect to my Fly.io URL

Cause: The URL is slightly different, or there's a typo.

Fix:

1. Go to your Fly.io dashboard: https://fly.io/dashboard
2. Find your app name in the list
3. Click on it and look for the URL (usually shown as `phishguard.fly.dev`)
4. Copy the exact URL
5. Update `background.js` and `popup.js` with the correct URL

I see "Mixed Content" error in Chrome DevTools

Cause: The extension is loaded over HTTPS but trying to reach HTTP.

Fix: Always use `https://` for your Fly.io URL (it should already be HTTPS by default).

The extension loads but `/predict` endpoint fails

Cause: The model file is missing on Fly.

Fix: Same as "health endpoint" troubleshooting above.

Summary of What You Did

✅ Pushed your code to GitHub
✅ Connected GitHub to Fly.io
✅ Deployed the ML backend to Fly.io (HTTPS endpoint)
✅ Updated extension to use Fly.io backend
✅ Tested the extension locally in Chrome

You're now ready to:

- Publish the extension to Chrome Web Store (optional), OR
- Keep using it locally with the Fly.io backend

Next Steps

Once you've verified everything works:

1. **Chrome Web Store Publishing** — I can help you prepare the package, privacy policy, and screenshots
2. **More Testing** — Visit various websites and test classification accuracy
3. **Gather Feedback** — See if the extension catches real phishing attempts

Let me know which step you want to do next!

DEPLOY_TO_RAILWAY.md

Deploy PhishGuard ML Backend to Railway (Step-by-Step)

Railway is **free** ($5/month credit), **easy**, and **fast** to deploy. Follow these steps.

Step 1: Go to Railway.app

1. Open https://railway.app in your browser
2. Click **"Start a New Project"** (or "Create")
3. You'll see options. Click **"Deploy from GitHub repo"**

Step 2: Connect Your GitHub Account

1. Click **"GitHub"** (sign in if needed)
2. GitHub will ask permission - click **"Authorize railway-app"**
3. You'll be redirected back to Railway

Step 3: Select Your Repository

1. You should see a list of your GitHub repos
2. **Find and click** `G-man312/Phishguard-` (the one we just pushed)
3. Click **"Deploy now"**

Step 4: Configure the Deployment

Railway will show you a deploy form. You need to:

4.1 Set the Service Name (optional but helpful)

- Change from default to: `phishguard-ml`
- This helps you identify the service later

4.2 Set the Root Directory

- Click **"Root Directory"** dropdown
- Change to: `ml-backend`
- **(This is important!)** This tells Railway where the Dockerfile is

4.3 Click "Deploy"

- Railway will start building (takes 1-3 minutes)
- You'll see a log showing the build progress

Step 5: Wait for Deployment

You'll see:

```
Building...
Running Dockerfile...
Installing dependencies...
✓ Deployment complete
```

Once done, Railway shows your app's details on the right side panel.

Step 6: Get Your Deployment URL

In the **right panel**, look for:

- **"Domains"** section
- You should see a URL like: `https://phishguard-ml-production-xxxx.railway.app`

Copy and save this URL! (You'll use it in 5 minutes)

Step 7: Test the Backend

1. Open your browser
2. Go to: `https://YOUR_RAILWAY_URL/health`
   - Replace `YOUR_RAILWAY_URL` with your actual URL from Step 6
   - Example: `https://phishguard-ml-production-xxxx.railway.app/health`
3. You should see JSON:

```json
{
  "status": "healthy",
```

```
  "model_loaded": true
}
```

If you see this ✅ → Your backend is live and working!

If you see an error ❌ → Check the Railway logs for errors

Troubleshooting

I don't see a URL in Railway

- Click on your deployment/service name in the left sidebar
- Look for "Domains" section on the right panel
- It might take 2-3 minutes to generate

`/health` returns an error

- Check Railway logs (there's a "Logs" tab)
- Look for the error message
- Most common: Model file missing (but it should be there since we pushed it to GitHub)

Build failed

- Check the build logs in Railway
- Make sure `ml-backend/Dockerfile` exists and is correct
- Redeploy by clicking the "Redeploy" button

Next Steps

Once you see the "healthy" response ✅:

1. **Copy your Railway URL**
2. Go back to VS Code
3. I'll update `background.js` and `popup.js` to use this URL
4. Test the extension in Chrome
5. Ready to publish!

Notes

- **You're on the free tier** ($5/month credit)
- No credit card charged (you won't exceed $5 with light testing)
- Your app will always be running (no cold starts like Render)
- HTTPS is automatic (Railway handles it)

MAINTENANCE_GUIDE.md

PhishGuard Maintenance Guide

How to Manual Update Site Lists

Currently, the "Database" of known Safe and Malicious sites is stored in **hardcoded arrays** within the extension files.

Because the extension runs in different contexts (Popup vs Background), these lists are currently **duplicated** in two files. You must update **BOTH** locations to ensure consistency.

1. Adding a Malicious Site (Blocklist)

Add the domain (e.g., `'example-phish.com'`) to the `MALICIOUS_DATASET` array in both:

1. **`phish-guard/classifier.js`** (~Line 60)

```
const MALICIOUS_DATASET = [
  'purplehoodie.com',
  'apunkagames.net',
  'YOUR-NEW-SITE.COM', // <--- Add here
  ...
];
```

2. **`phish-guard/background.js`** (Inside `classifyQuick` function, ~Line 622)
   - *Note: This is an inline backup list.*

```
const inMaliciousDataset = (self._PG_MALICIOUS_DATASET || [..., 'apunkagames.net',
'YOUR-NEW-SITE.COM', ...]).some(...)
```

---

2. Adding a Safe Site (Whitelist)

Add the domain to the `SAFE_DATASET` array in both:

1. **`phish-guard/background.js`** (~Line 74)

```
const SAFE_DATASET = [
  'google.com',
  'YOUR-SAFE-SITE.COM', // <--- Add here
  ...
];
```

2. **`phish-guard/classifier.js`** (~Line 5)

```
const SAFE_DATASET = [
  'google.com',
  'YOUR-SAFE-SITE.COM', // <--- Add here
  ...
];
```

3. Updating the ML Model (Advanced)

If you want the AI to learn new patterns, you need to:

1. Add the URL and label (`good` or `bad`) to `ml-backend/datasets/training_data.csv` (if it exists) or `URLDataset.csv` .
2. Run `python train_model.py` to regenerate the `phishguard_model.pkl` .
3. Redeploy the backend (Railway automatically handles this when you push).

ROADMAP.md

PhishGuard Development Roadmap

✅ Completed (25% of Project)

Phase 1: Core Extension (COMPLETED)

- ✅ Chrome Extension Manifest V3 architecture
- ✅ Four-label classification system (`safe` , `unknown` , `suspicious` , `malicious`)
- ✅ URL heuristics (IP addresses, subdomains, hyphens, punycode, HTTPS, @ symbol, keywords)
- ✅ Safe and malicious domain datasets
- ✅ Dynamic toolbar icons (green/yellow/red/gray)
- ✅ Full-screen malicious site interstitial
- ✅ Suspicious site banner/notification
- ✅ Whitelist/blacklist management
- ✅ Dark/Light mode toggle (synced across extension)
- ✅ Popup UI with action buttons

Phase 2: Machine Learning Integration (COMPLETED)

- ✅ Python Flask API backend
- ✅ 18-feature extraction (URL + webpage characteristics)
- ✅ Random Forest Classifier training
- ✅ Training on 11,449 real-world samples (83.76% accuracy)
- ✅ Hybrid classification (URL heuristics + ML predictions)
- ✅ Parallel data collection optimization
- ✅ Model serving endpoint (`/predict`)

🚀 Next Steps (Remaining 75% of Project)

Phase 3: Testing & Validation (IMMEDIATE - Next 1-2 Days)

3.1 Model Testing & Validation

- ☐ Test ML model with complex phishing URLs (similar to training dataset)
  - ☐ Test URLs with multiple subdomains
  - ☐ Test URLs with suspicious keywords
  - ☐ Test URLs with @ symbols and hyphens
  - ☐ Test IP addresses
  - ☐ Verify ML predictions match expected behavior
- ☐ Integration testing with Chrome extension
  - ☐ Verify ML API calls from `background.js` work correctly
  - ☐ Test hybrid classification logic (heuristics + ML)

    - [ ] Verify icon colors update based on ML predictions
    - [ ] Test suspicious/malicious popups trigger correctly
- [ ] Performance testing
    - [ ] Measure ML API response time
    - [ ] Test with slow network connections
    - [ ] Verify graceful fallback when ML API is unavailable
- [ ] Accuracy validation
    - [ ] Test false positives (legitimate sites marked suspicious)
    - [ ] Test false negatives (phishing sites missed)
    - [ ] Collect feedback on misclassifications

3.2 Model Improvements

- [ ] Retrain with additional datasets for better accuracy
    - [ ] Add more legitimate URLs to balance dataset
    - [ ] Add recent phishing URLs (2024-2025) to stay current
    - [ ] Target accuracy >90%
- [ ] Feature engineering improvements
    - [ ] Add SSL certificate validation features
    - [ ] Add domain age/registration features
    - [ ] Add redirect chain analysis
    - [ ] Add meta tag analysis (title, description keywords)
- [ ] Model architecture optimization
    - [ ] Experiment with different algorithms (XGBoost, Neural Networks)
    - [ ] Hyperparameter tuning
    - [ ] Cross-validation for better generalization

Phase 4: Backend Server Infrastructure (Short-term - 1-2 Weeks)

4.1 Production Backend Setup

- [ ] Deploy Flask API to cloud service (AWS, GCP, Azure, or Heroku)
    - [ ] Set up production environment
    - [ ] Configure environment variables
    - [ ] Set up SSL/TLS certificates
    - [ ] Configure CORS for extension
- [ ] Database integration
    - [ ] Set up PostgreSQL/MySQL for storing:
        - Known malicious domains (expandable database)
        - User reports
        - Community moderation data
    - [ ] Design database schema
    - [ ] Implement CRUD operations
- [ ] API enhancements

    - [ ] Add authentication/API keys
    - [ ] Rate limiting for API requests
    - [ ] Request logging and analytics
    - [ ] Health monitoring and alerting
    - [ ] Caching layer (Redis) for frequent queries

4.2 Malicious Domain Database Expansion

- [ ] Integrate with threat intelligence feeds
    - [ ] Phishtank API integration
    - [ ] Google Safe Browsing API
    - [ ] VirusTotal API
    - [ ] OpenPhish feed
- [ ] Real-time database updates
    - [ ] Automated feed ingestion
    - [ ] Periodic database refresh (hourly/daily)
    - [ ] Verify domain status before adding to database
- [ ] Database query optimization
    - [ ] Fast domain lookup (hash tables, bloom filters)
    - [ ] Batch queries for multiple domains

Phase 5: Community Moderation System (Medium-term - 2-3 Weeks)

5.1 User Reporting System

- [ ] Report submission interface
    - [ ] "Report Site" button in extension popup (for suspicious/unknown sites)
    - [ ] Report form with:
        - Site URL
        - Category (phishing, malware, scam, false positive)
        - Optional description/comments
        - User email (optional, for follow-up)
    - [ ] Submit report to backend API
- [ ] Report storage and tracking
    - [ ] Store reports in database
    - [ ] Generate unique report IDs
    - [ ] Track report status (pending, reviewed, resolved, dismissed)
    - [ ] User dashboard to view their reports
- [ ] Report validation
    - [ ] Auto-check against existing database
    - [ ] Flag duplicate reports
    - [ ] Basic spam/abuse detection

5.2 Moderation Dashboard

- [ ] Admin/moderation panel

    - [ ] View pending reports
    - [ ] Review reported sites
    - [ ] Manual classification override
    - [ ] Bulk actions (approve/dismiss multiple reports)
- [ ] Moderator tools
    - [ ] Site verification tool (quick check interface)
    - [ ] Notes/annotations for sites
    - [ ] History tracking (who reviewed, when, decision)
- [ ] Automated moderation rules
    - [ ] Auto-approve reports with high confidence
    - [ ] Escalate complex cases to human moderators
    - [ ] Threshold-based decisions (e.g., 10+ reports = auto-flag)

5.3 Community Features

- [ ] User reputation system
    - [ ] Track accurate reports
    - [ ] Reward trusted reporters
    - [ ] Penalize false reports
- [ ] Public transparency
    - [ ] Public list of verified safe sites
    - [ ] Public list of verified malicious sites
    - [ ] Statistics dashboard (sites reported, moderation activity)
- [ ] Feedback loop
    - [ ] Notify users when their report is reviewed
    - [ ] Allow users to dispute moderation decisions
    - [ ] Community voting on edge cases

Phase 6: Advanced Features (Long-term - 3-4 Weeks)

6.1 Enhanced Detection

- [ ] Screenshot-based analysis
    - [ ] Capture page screenshots
    - [ ] OCR for text extraction
    - [ ] Visual similarity to known phishing pages
- [ ] JavaScript behavior analysis
    - [ ] Detect suspicious JavaScript patterns
    - [ ] Form submission tracking
    - [ ] External data exfiltration detection
- [ ] Multi-factor validation
    - [ ] Combine ML, heuristics, database, and community reports
    - [ ] Confidence scoring system

    - [ ] Adaptive thresholds based on risk level

6.2 User Experience Enhancements

- [ ] Settings page improvements
    - [ ] Sensitivity slider (more/less aggressive detection)
    - [ ] Custom notification preferences
    - [ ] Whitelist/blacklist bulk import/export
- [ ] Educational features
    - [ ] Show why a site was flagged (detailed breakdown)
    - [ ] Tips for identifying phishing sites
    - [ ] Security best practices
- [ ] Multi-language support
    - [ ] Translate extension UI
    - [ ] Localize phishing detection for different regions

6.3 Analytics & Insights

- [ ] Extension analytics
    - [ ] Track sites classified per day
    - [ ] Most common suspicious patterns
    - [ ] User engagement metrics
- [ ] Threat intelligence dashboard
    - [ ] Trending malicious domains
    - [ ] Geographic distribution
    - [ ] Attack patterns and trends

Phase 7: Production Readiness (Final Phase)

7.1 Security & Privacy

- [ ] Privacy policy
    - [ ] Document what data is collected
    - [ ] Explain how data is used
    - [ ] User consent mechanisms
- [ ] Security audit
    - [ ] Code review for vulnerabilities
    - [ ] Penetration testing
    - [ ] Secure API endpoints
- [ ] Data protection
    - [ ] Encrypt sensitive data
    - [ ] Implement GDPR compliance
    - [ ] User data deletion capabilities

7.2 Deployment & Distribution

- ☐ Chrome Web Store submission
  - ☐ Prepare store listing (screenshots, description)
  - ☐ Privacy policy and terms of service
  - ☐ Submit for review
- ☐ Beta testing program
  - ☐ Recruit beta testers
  - ☐ Collect feedback
  - ☐ Iterate based on feedback
- ☐ Documentation
  - ☐ User guide
  - ☐ Developer documentation
  - ☐ API documentation
  - ☐ Troubleshooting guide

7.3 Maintenance & Updates

- ☐ Automated testing
  - ☐ Unit tests for core functionality
  - ☐ Integration tests
  - ☐ End-to-end tests
- ☐ Continuous improvement
  - ☐ Regular model retraining with new data
  - ☐ Monitor false positive/negative rates
  - ☐ Update threat intelligence feeds
  - ☐ Patch security vulnerabilities

📊 Progress Tracking

Overall Progress: ~25% Complete

- ✅ **Phase 1 (Core Extension)**: 100% Complete
- ✅ **Phase 2 (ML Integration)**: 100% Complete
- ⏳ **Phase 3 (Testing & Validation)**: 0% Complete - **START HERE**
- ⏳ **Phase 4 (Backend Infrastructure)**: 0% Complete
- ⏳ **Phase 5 (Community Moderation)**: 0% Complete
- ⏳ **Phase 6 (Advanced Features)**: 0% Complete
- ⏳ **Phase 7 (Production Readiness)**: 0% Complete

🎯 Immediate Action Items (Next Session)

1. **Test ML Model** (30 minutes)
   - Run Flask server: `cd ml-backend && python app.py`
   - Test complex phishing URLs from the dataset
   - Verify predictions make sense
2. **Integration Testing** (1 hour)

- Verify Chrome extension calls ML API correctly
- Test hybrid classification logic
- Check icon colors and popups work with ML predictions

3. **Gather Feedback** (30 minutes)
   - Test on real websites
   - Note any false positives/negatives
   - Document issues for model improvement

📝 Notes

- **ML Model**: Currently 83.76% accuracy on 11,449 samples. Target: >90%
- **API Endpoint**: `http://localhost:5000/predict` (currently local, needs cloud deployment)
- **Database**: Not yet implemented (Phase 4)
- **Community Features**: Not yet implemented (Phase 5)

Last Updated: After Phase 2 completion Next Review: After Phase 3 testing

---

TESTING_THEORY.md

PhishGuard Testing Theory

5.3 Testing

5.3.1 Unit Testing

Unit testing involves verifying individual program components or functions in isolation from the rest of the system. For PhishGuard, this ensures that smaller sections of code work as intended, such as the URL feature extraction function (`extract_features_for_prediction()`), URL classification logic (`classifyQuick()`), or individual ML model prediction methods. Each function is tested independently with controlled inputs and expected outputs to detect early errors and ensure building blocks perform their specific tasks correctly before being combined into the larger system.

5.3.2 Integration Testing

Integration testing verifies that individual modules or components of the system work together cohesively. After unit tests, integration testing ensures that the Chrome extension's background script (`background.js`) interacts correctly with the Flask API server (`app.py`), and that the ML model can seamlessly receive feature vectors, process them, and return predictions. It also validates data flow between the extension popup (`popup.js`) and the background service worker, identifying interface issues, data mismatches, and communication failures between connected modules or external systems.

5.3.3 Blackbox Testing

Blackbox testing examines the overall system functionality without considering its internal code structure or implementation details. A tester provides input data (e.g., various URLs including safe, suspicious, and malicious sites) and evaluates if the system produces the expected outputs (e.g., correct icon color changes, accurate classification labels, appropriate ML probability scores). The focus is entirely on input-output behavior, ensuring the PhishGuard

extension performs correctly from the user's perspective and meets the objective of reliably detecting phishing websites with high accuracy.

TEST_SITES.md

PhishGuard Test Sites

Use these URLs to verify the extension's classification logic.

🟢 Safe Sites (Green Badge)

These sites are in the `SAFE_DATASET` (Allowlist). They should always appear as **Safe**.

- Google
- GitHub
- Railway
- Drive
- PayPal (Real site)

🔴 Malicious Sites (Red Badge)

These sites are in the `MALICIOUS_DATASET` (Blocklist). They should always appear as **Malicious**.

- `https://purplehoodie.com`
- `https://apunkagames.net`
- `https://sadeempc.com`
- `https://crackingpatching.com`
- `https://oceanofgames.com`

🟠 Suspicious Sites (Orange Badge)

These URLs trigger specific **Heuristic Checks** in our code.

Trigger	Test URL (Copy & Paste)	Why?
Brand Misuse	`http://paypal-secure-update.com`	Contains "paypal" but is not `paypal.com`
Brand Misuse	`http://google-login-verify.net`	Contains "google" but is not `google.com`
Risky TLD	`http://example-test.xyz`	Ends in `.xyz` (Risky TLD)
Risky TLD	`http://bank-login.tk`	Ends in `.tk` (Risky TLD)
IP Address	`http://192.168.1.1`	Hostname is an IP address
Keywords	`http://example.com/login-secure-update`	contains "login", "secure", "update"
@ Symbol	`http://user:pass@example.com`	Contains `@` (Auth bypass attempt)
Not HTTPS	`http://example.com`	Plain HTTP

> ***Note:*** *Some of these might not resolve to a real page, but the extension will still analyze the URL in the address bar.*

🟣 Unknown Sites (Grey Badge)

These sites are **Safe** but **Not in our Allowlist**. They should appear as "Unknown". (Unless the ML model flags them as suspicious remotely).

> ***Note:*** *Our ML model prefers* `www.` *subdomains. Use the full URLs below.*

- `https://www.rust-lang.org`
- `https://www.etherscan.io`
- `https://www.neocities.org`
- `https://www.projecteuler.net`
- `https://www.lichess.org`
- `https://www.example.com`